

Character Classes

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>.</code>	Any character except newline EXAMPLE: <code>a.c</code> → <code>'abc'</code> , <code>'a1c'</code>
<code>\w</code>	Word character: <code>[a-zA-Z0-9_]</code> EXAMPLE: <code>\w+</code> → <code>'hello'</code> , <code>'world_1'</code>
<code>\W</code>	Non-word character EXAMPLE: <code>\W</code> → <code>' '</code> , <code>'!'</code>
<code>\d</code>	Digit: <code>[0-9]</code> EXAMPLE: <code>\d{3}</code> → <code>'123'</code>
<code>\D</code>	Non-digit EXAMPLE: <code>\D+</code> → <code>'abc'</code>
<code>\s</code>	Whitespace: space, tab, newline EXAMPLE: <code>a\s b</code> → <code>'a b'</code>
<code>\S</code>	Non-whitespace EXAMPLE: <code>\S+</code> → <code>'hello'</code>
<code>[abc]</code>	Character set — matches a, b, or c EXAMPLE: <code>[aeiou]</code> → <code>'a'</code> , <code>'e'</code>
<code>[^abc]</code>	Negated set — matches anything not in the set EXAMPLE: <code>[^aeiou]</code> → <code>'b'</code> , <code>'c'</code>
<code>[a-z]</code>	Range — matches any character from a to z EXAMPLE: <code>[a-f]+</code> → <code>'abc'</code>
<code>[a-zA-Z]</code>	Combined range — any letter EXAMPLE: <code>[a-zA-Z]+</code> → <code>'Hello'</code>

Anchors

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>^</code>	Start of string (or line with multiline flag) EXAMPLE: <code>^Hello</code> → 'Hello world'
<code>\$</code>	End of string (or line with multiline flag) EXAMPLE: <code>world\$</code> → 'Hello world'
<code>\b</code>	Word boundary EXAMPLE: <code>\bcata\b</code> → 'cat' not 'catch'
<code>\B</code>	Non-word boundary EXAMPLE: <code>\Bcata\B</code> → 'concatenate'
<code>\A</code>	Start of string (no multiline) EXAMPLE: <code>\AHello</code> → 'Hello world'
<code>\Z</code>	End of string (no multiline) EXAMPLE: <code>world\Z</code> → 'Hello world'

Quantifiers

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>*</code>	0 or more (greedy) EXAMPLE: <code>ab*</code> → <code>'a'</code> , <code>'ab'</code> , <code>'abbb'</code>
<code>+</code>	1 or more (greedy) EXAMPLE: <code>ab+</code> → <code>'ab'</code> , <code>'abbb'</code>
<code>?</code>	0 or 1 (optional) EXAMPLE: <code>ab?</code> → <code>'a'</code> , <code>'ab'</code>
<code>{n}</code>	Exactly n times EXAMPLE: <code>a{3}</code> → <code>'aaa'</code>
<code>{n,}</code>	n or more times EXAMPLE: <code>a{2,}</code> → <code>'aa'</code> , <code>'aaa'</code>
<code>{n,m}</code>	Between n and m times EXAMPLE: <code>a{2,4}</code> → <code>'aa'</code> , <code>'aaa'</code> , <code>'aaaa'</code>
<code>*?</code>	0 or more (lazy / non-greedy) EXAMPLE: <code><.*?></code> → <code>'<a>'</code> not <code>'<a>b'</code>
<code>+?</code>	1 or more (lazy) EXAMPLE: <code>a+?</code> → first <code>'a'</code> only
<code>??</code>	0 or 1 (lazy) EXAMPLE: <code>ab??</code> → <code>'a'</code> preferred

Groups & Lookarounds

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
<code>(abc)</code>	Capturing group EXAMPLE: <code>(foo)bar</code> → captures 'foo'
<code>(?:abc)</code>	Non-capturing group EXAMPLE: <code>(?:foo)bar</code> → groups but doesn't capture
<code>(?<name>abc)</code>	Named capturing group EXAMPLE: <code>(?<year>\d{4})</code> → named capture 'year'
<code>(?=abc)</code>	Positive lookahead EXAMPLE: <code>foo(=bar)</code> → 'foo' before 'bar'
<code>(?!abc)</code>	Negative lookahead EXAMPLE: <code>foo(!bar)</code> → 'foo' not before 'bar'
<code>(?<=abc)</code>	Positive lookbehind EXAMPLE: <code>(?<=foo)bar</code> → 'bar' after 'foo'
<code>(?<!abc)</code>	Negative lookbehind EXAMPLE: <code>(?<!foo)bar</code> → 'bar' not after 'foo'
<code>\1</code>	Backreference to group 1 EXAMPLE: <code>(\w)\1</code> → 'aa', 'bb'
<code>(?:a b)</code>	Alternation inside group EXAMPLE: <code>(?:cat dog)</code> → 'cat' or 'dog'

Flags / Modifiers

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
i	Case-insensitive matching EXAMPLE: <code>/hello/i</code> → 'Hello', 'HELLO'
g	Global — find all matches, not just first EXAMPLE: <code>/a/g</code> → all 'a's in string
m	Multiline — ^ and \$ match line starts/ends EXAMPLE: <code>/^foo/m</code> → 'foo' at start of any line
s	Dotall — . matches newline too EXAMPLE: <code>/a.b/s</code> → 'a\nb'
u	Unicode — enable full Unicode support EXAMPLE: <code>/\u{1F600}/u</code> → emoji match
y	Sticky — match at exact position only EXAMPLE: <code>/foo/y</code> at index 0
x	Extended — allow whitespace and comments (some engines) EXAMPLE: <code># comment</code>